Centro Universitário de Excelência
Sistemas de Informação

MecâniQA: Web API Fundacional

Autor: Kevilyn Bitencourt da Cruz
Hélio Nunes Moraes Junior
Rodrigo Alves Oliveira
Marcelo Henrique Fernandes Mauricio
Murillo Rigaud dos Santos Oliveira

Vitória da Conquista, 2026

Agenda

Apresentar o problema, os conceitos de Orientação a Objetos e a construção da Web API fundacional da MecâniQA.

1. Contexto e conceitos de OO:
Problema, MVP e modelagem das classes

2. Controllers e API REST:
Rotas, endpoints, verbos e status HTTP

3. Padrão Singleton:
Instância única e estado em memória

4. Resultados e conclusão:
CRUDs, testes e limitações

5. Referências:
Fontes utilizadas no projeto

Contexto e conceitos de OO

- Problema: modernizar o catálogo da MecâniQA por meio de uma Web API.
- Objetivo do MVP: executar o CRUD de Peças e Serviços em memória.
- Peca e Servico agrupam atributos private e seus construtores.
- static pertence à classe; CategoriaPeca restringe as categorias.
- Getters e setters controlam o acesso e a alteração dos atributos.

Controllers e API REST

- @RestController recebe requisições e devolve JSON.
  - @RequestMapping define a rota-base; os mappings criam endpoints.
  - POST cria; GET consulta; PUT atualiza; DELETE remove.
  - Retornos: 201 criado; 200 sucesso; 204 sem corpo; 404 ausente.

```java
@RestController
@RequestMapping("/api/pecas")
public class PecaController {
    private final PecaRepository repository;

    public PecaController() {
        this.repository = PecaRepository.getInstance();
    }

    @PostMapping
    public ResponseEntity<Peca> cadastrar(@RequestBody Peca peca) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(repository.salvar(peca));
    }

    @GetMapping("/{codigo}")
    public ResponseEntity<Peca> buscar(@PathVariable Long codigo) {
        return repository.buscarPorCodigo(codigo)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

Padrão Singleton

- instance é static e pertence à própria classe.
    - O construtor private impede new fora do repository.
    - getInstance() cria uma vez e devolve a mesma instância.
    - A lista preserva o estado enquanto o servidor está ativo.



```java
public class PecaRepository {
    private static PecaRepository instance;
    private final List<Peca> pecas;

    private PecaRepository() {
        this.pecas = new ArrayList<>();
    }

    public static PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}
```

Resultados e conclusão

- Os CRUDs de Peças e Serviços atendem às oito User Stories.
- Códigos e datas são gerados automaticamente pela aplicação.
- O Postman possui 18 requisições e o JUnit valida o módulo de Peças.
- A API utiliza memória, sem banco, Spring Data ou @Autowired.
- Ao reiniciar o servidor, as listas e os identificadores são reiniciados.

Referências

- MECÂNIQA. OAT 1 - Construindo a Web API Fundacional da MecâniQA. 2026.
- EQUIPE PORTO ALEGRE. mecaniQA-api-porto-alegre. GitHub, 2026.
- github.com/Meizolas/mecaniQA-api-porto-alegre
- SPRING. Spring Boot Reference Documentation.
- docs.spring.io/spring-boot
- ORACLE. Java 17 Documentation.
- docs.oracle.com/en/java/javase/17